

机器视觉实验报告

专业 智能科学与技术 姓名 史峰源 学号 2412526

1 实验目的

1. 理解并掌握图像分割与二值化处理的基本原理，熟练利用 C++ 与 OpenCV 实现基于灰度直方图的 p 率阈值化算法。
2. 深入理解二值图像的拓扑性质，掌握形态学与边界处理的概念，编写代码实现基于 Moore 邻域的图像内边界跟踪算法。
3. 提升图像处理算法的底层编写能力，掌握像素级遍历、直方图统计以及相对路径下图像文件的读写操作。

2 实验原理

2.1 p 率阈值化（利用直方图）

p 率阈值化 (P-tile thresholding) 是一种基于先验知识的全局阈值分割方法。其基本假设是：如果已知图像中目标物体所占图像总面积的比例（即 p 率，常记为 p ），则可以通过灰度直方图来确定二值化的阈值 T 。

具体原理为：首先统计整幅图像的灰度直方图，得到各个灰度级的像素频数。设图像总像素数为 N ，目标物体面积占比为 p 。若目标物体比背景亮，则从最大灰度级（255）开始向最小灰度级（0）累加像素数量；当累加的像素总数首次大于或等于 $N \times p$ 时，对应的灰度级即为所求的阈值 T 。随后，利用该阈值 T 对原图像进行二值化操作，大于等于 T 的像素设为前景（255），小于 T 的设为背景（0）。

2.2 内边界跟踪

内边界跟踪主要用于提取二值图像中连通区域的外侧轮廓，通常采用 Moore 邻域边界跟踪算法。其核心思想是：从左至右、从上至下扫描图像，当遇到一个前景像素（灰度值为 255），且其左侧相邻像素为背景（灰度值为 0）时，将该前景像素作为边界追踪的起点。以该起点为中心，按照顺时针（或逆时针）方向检查其 8 邻域的像素点。当找到第一个前景像素时，将其作为下一个边界点，并以该点为新中心，更新探测的起始方向，继续顺时针探测，直到当前追踪的中心像素重新回到初始起点，即表示该连通区域的内边界跟踪完成。这种方法能有效地将目标区域的轮廓提取出来，用于后续的形状特征分析。

3 实验步骤

1. **环境配置与工程搭建**：在 IDE（如 Visual Studio）中创建 C++ 工程，配置好 OpenCV 库的包含目录与附加依赖项。
2. **读取图像**：将测试图片（testing.jpg）放入 C++ 工程文件所在的相对路径下，利用 `imread` 读取原图。
3. **彩色转灰度**：利用 `cvtColor` 将读取的彩色图像转化为单通道灰度图像，以便进行后续处理。
4. **执行 p 率阈值化**：
 - 编写 `myThresholdP` 函数，遍历灰度图统计 0 255 的像素直方图。
 - 设定先验比例 p （如 0.3），计算目标像素数。

- 根据目标亮度从 255 向 0（或 0 向 255）累加，寻找满足累加值 $\geq$ 目标像素数的阈值。
- 遍历原图应用该阈值，完成二值化处理。

5. 执行内边界跟踪：

- 先利用 `threshold` 函数结合 `THRESH_OTSU` 方法将灰度图转化为标准的二值图像（只有 0 和 255）。
- 编写 `myInnerContours` 函数，采用 Moore 邻域顺时针 8 方向探寻法，遍历整个二值图，对每一个未被标记过的左侧为背景的前景像素点发起循环跟踪，直至回到起点。
- 将遍历过程中的边界像素在空白图上标为 255，从而提取出最终的内边界图。

6. **结果展示与输出：**调用 `imshow` 函数同时展示输入原图、灰度图与处理后的结果图，观察并验证算法执行效果。

4 程序代码

注：根据实验要求，代码中已统一使用相对路径读取图片。

4.1 p 率阈值化 (C++)

```
#include <iostream>
#include <fstream>
#include "opencv2/opencv.hpp"
#include "opencv2/imgproc/imgproc.hpp"
#include "opencv2/highgui/highgui.hpp"
#include <stdio.h>
using namespace cv;
using namespace std;

Mat myThresholdP(Mat img)
{
    // 初始化与原图同等大小和类型的全黑图像
    Mat ThresholdPImg = Mat::zeros(img.size(), CV_8UC1);

    /*
    p率阈值化代码实现
    */

    // 1. 统计图像的灰度直方图
    int hist[256] = {0};
    int total_pixels = img.rows * img.cols;
    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            hist[img.at<uchar>(i, j)]++;
        }
    }

    // 2. 设定目标物体的面积占比 p 率
    double p = 0.4;
```

```

int target_pixels = total_pixels * p;

// 3. 根据p率寻找阈值
int sum = 0;
int threshold_value = 0;
for (int i = 255; i >= 0; i--) {
    sum += hist[i];
    if (sum >= target_pixels) {
        threshold_value = i;
        break;
    }
}

// 4. 应用求得的阈值进行二值化
for (int i = 0; i < img.rows; i++) {
    for (int j = 0; j < img.cols; j++) {
        if (img.at<uchar>(i, j) >= threshold_value) {
            ThresholdPImg.at<uchar>(i, j) = 255;
        } else {
            ThresholdPImg.at<uchar>(i, j) = 0;
        }
    }
}

return ThresholdPImg;
}

void main()
{
    Mat input = imread("testing.jpg");
    if (input.empty()) {
        cout << "无法读取图像，请确认相对路径下存在 testing.jpg 文件！" << endl;
        return;
    }

    Mat gray;
    //彩色图转为灰度图
    cvtColor(input, gray, COLOR_BGR2GRAY);

    //灰度图p率阈值化代码的实现
    Mat ThresholdPImg = myThresholdP(gray);

    imshow("input", input);
    imshow("gray", gray);
    imshow("ThresholdPImg", ThresholdPImg);
    waitKey(0);
}

```

4.2 内边界跟踪 (C++)

```
#include <iostream>
#include <fstream>
#include "opencv2/opencv.hpp"
#include "opencv2/imgproc/imgproc.hpp"
#include "opencv2/highgui/highgui.hpp"
#include <stdio.h>

using namespace cv;
using namespace std;

Mat myInnerContours(Mat img)
{
    // 初始化为与原图大小相同的全黑图像
    Mat InnerContoursImg = Mat::zeros(img.size(), CV_8UC1);

    /*
    编写二值图像内边界跟踪的代码实现
    */

    Mat visited = Mat::zeros(img.size(), CV_8UC1);

    int dx[8] = { 1, 1, 0, -1, -1, -1, 0, 1 };
    int dy[8] = { 0, 1, 1, 1, 0, -1, -1, -1 };

    // 遍历图像，寻找边界的起始点
    // 为了防止越界，略过最外层一圈像素
    for (int i = 1; i < img.rows - 1; i++) {
        for (int j = 1; j < img.cols - 1; j++) {

            if (img.at<uchar>(i, j) == 255 && img.at<uchar>(i, j - 1) == 0 && visited.at<uchar>(i, j) ==
                0) {

                int startX = j;
                int startY = i;
                int currX = startX;
                int currY = startY;

                // 初始搜索方向：由于左侧是背景，故从左上方向(即索引5)开始顺时针搜索下一个前景点
                int dir = 5;

                // 开始Moore邻域轮廓跟踪
                do {
                    InnerContoursImg.at<uchar>(currY, currX) = 255;
                    visited.at<uchar>(currY, currX) = 255;

                    bool found = false;
                    for (int k = 0; k < 8; k++) {
```

```

        int searchDir = (dir + k) % 8;
        int nx = currX + dx[searchDir];
        int ny = currY + dy[searchDir];

        if (nx >= 0 && nx < img.cols && ny >= 0 && ny < img.rows) {
            if (img.at<uchar>(ny, nx) == 255) {
                currX = nx;
                currY = ny;

                dir = (searchDir + 5) % 8;
                found = true;
                break;
            }
        }

        if (!found) {
            break;
        }

        } while (currX != startX || currY != startY);
    }
}

return InnerContoursImg;
}

void main()
{
    Mat input = imread("testing.jpg");

    Mat gray;
    // 彩色图转为灰度图
    cvtColor(input, gray, COLOR_BGR2GRAY);

    // 灰度图转为二值图，使用大津法(OTSU)自动寻找阈值
    Mat binary;
    threshold(gray, binary, 0, 255, THRESH_OTSU);

    // 二值图像内边界跟踪的代码实现
    Mat InnerContoursImg = myInnerContours(binary);

    imshow("input", input);
    imshow("gray", gray);
    imshow("binary", binary);
    imshow("InnerContoursImg", InnerContoursImg);
    waitKey(0);
}

```

5 实验结果显示

5.1 p 率阈值化实验结果



图 1: p 率阈值化处理结果

5.2 内边界跟踪实验结果



图 2: 内边界跟踪处理结果

6 实验分析总结

1. **对 p 率阈值化算法的分析：**该算法由于依赖目标物体占比面积这一个先验信息，对于具有固定占比场景的工业检测非常有效且速度较快。但在未知目标大小，或者同类图片中目标所占比例变化较大的情况时，该算法不能做到自适应提取。通过手动调节代码中 p 值的比例，可以观察到由于提取阈值上下波动造成的噪点或图像缺损。
2. **对内边界跟踪算法的分析：**采用 Moore 邻域算法跟踪能够稳定、连贯地获取图像中各个联通域的内边界。需要着重注意起始点条件（左侧必须为背景）以及搜索方向迭代时的偏置值计算（ $\text{dir} = (\text{searchDir} + 5)$ ）

3. **实验总结：**通过本次实验，我不仅加强了 C++ 基于 OpenCV 矩阵遍历运算的编码能力，同时也深刻体会了数字图像处理中基于底层像素的算法运行逻辑。对于相对路径和工程部署也有了更为规范的认识，能够确保代码具有高可移植性。